# AI FOR SCIENCE:
# PUTTING PHYSICS INTO LEARNING

Introductory lecture

**Hadrien Vroylandt, Junior Professor**

Université Caen Normandie, ENSICAEN, CNRS, Normandie Univ, GREYC UMR 6072, F-14000 Caen, France

AISSAI Hackathon: AI for Particle Accelerators, 12 October 2026

hadrien.vroylandt@unicaen.fr

GREYC
Electronics and Computer Science Laboratory

Normandie Université

UNICAEN

ENSI CAEN
ÉCOLE PUBLIQUE D'INGÉNIEURS
CENTRE DE RECHERCHE

cnrs



# What AI has made possible

# Problems solved in the last few years

**Protein structure**

3D structure from the sequence alone, close to experimental accuracy

**Molecular simulation**

quantum-chemistry accuracy at force-field cost

**Weather forecasting**

10-day global forecast in a minute, as good as the best physics models

**Exoplanet detection**

a network finds a planet hidden in 4 years of Kepler light curves

**Materials discovery**

hundreds of thousands of new crystals predicted stable

**Maths & algorithms**

faster matrix multiplication, new combinatorial constructions

# Problems solved in the last few years

**Protein structure**

3D structure from the sequence alone, close to experimental accuracy

**AlphaFold 2 (2020)**

**Molecular simulation**

quantum-chemistry accuracy at force-field cost

**Deep Potential (2018)**
**MACE (2022)**

**Weather forecasting**

10-day global forecast in a minute, as good as the best physics models

**GraphCast (2023)**

**Exoplanet detection**

a network finds a planet hidden in 4 years of Kepler light curves

**Shallue & Vanderburg (2018)**

**Materials discovery**

hundreds of thousands of new crystals predicted stable

**GNoME (2023)**

**Maths & algorithms**

faster matrix multiplication, new combinatorial constructions

**AlphaTensor (2022)**
**FunSearch (2023)**

AlphaFold: Nature 596, 583 (2021) · DeePMD: PRL 120, 143001 (2018) · MACE: NeurIPS (2022) · GraphCast: Science 382, 1416 (2023) · Kepler-90i: AJ 155, 94 (2018) · GNoME: Nature 624, 80 (2023) · AlphaTensor: Nature 610, 47 (2022) · FunSearch: Nature 625, 468 (2024)

# What did AI improve?

## Speed

an expensive calculation becomes cheap

**Molecular simulation**
DFT → ML potential: $10^3$–$10^6\times$ faster

**Weather**
hours on a supercomputer → one minute on one chip

## Search

explore a huge, high-dimensional space

**Materials**
combinatorial space of compositions and structures

**Maths & algorithms**
space of programs, too large to enumerate

## Unknown map

learn a relation no theory gives in a usable form

**Protein structure**
sequence → 3D structure: seconds to hours instead of months to years of experiments

## Finding signals

rare, weak patterns in huge datasets

**Exoplanets**
trained on $\sim 15\,000$ labelled Kepler signals, then searched new stars: found Kepler-90i, the 8th planet of its system

# Why did these work?

**Data**

large curated databases built over decades (Protein Data Bank, weather reanalyses), or a simulator that produces labels on demand (DFT)

**A way to check**

we can tell whether a prediction is right: blind tests (CASP), comparison with what happened, a reference calculation, a proof

**Known structure**

symmetries, locality, conservation laws built into the model (rotation symmetry in AlphaFold and MACE)

- All of these models use prior knowledge of the problem, in addition to data
- In science we usually know a lot beforehand: **equations, symmetries, conservation laws**
- **This lecture: how to put this knowledge into a learned model**

# Science for AI: the other direction

| Concept from science | | AI method |
|---|---|---|
| Langevin dynamics, non-equilibrium thermodynamics | → | **diffusion models** (2015, 2020) |
| Boltzmann distribution, spin models | → | energy-based models, Boltzmann machines |
| Monte Carlo (1953); Hybrid Monte Carlo from lattice QCD (1987) | → | MCMC and HMC for Bayesian inference |
| variational free energy, mean-field theory | → | variational inference, VAEs (ELBO = −free energy) |
| optimal transport (Monge, Kantorovich) | → | Wasserstein GANs, flow matching |
| group theory, symmetries | → | equivariant networks (Part II) |

Many ML methods come from physics and statistics.

Sohl-Dickstein et al., ICML (2015) · Ho et al., NeurIPS (2020) · Metropolis et al., J. Chem. Phys. 21, 1087 (1953) · Duane et al., Phys. Lett. B 195, 216 (1987)

# ML in 5 minutes

# Learning = fitting a function

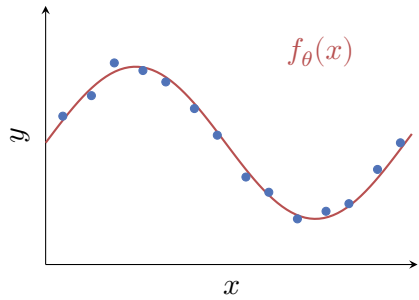
- Data: pairs $(x_i, y_i)$, $i = 1 \dots N$
- Model: a flexible function $f_\theta(x)$ with parameters $\theta$
- Loss: how wrong we are

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i),\, y_i)$$

- Optimisation: gradient descent

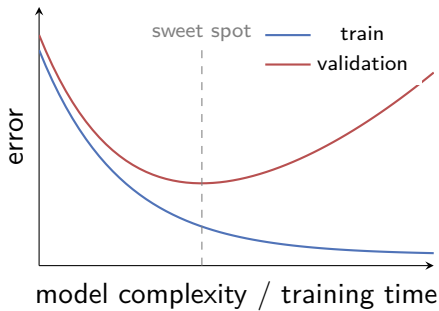
$$\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}$$

- $f_\theta$ is often a **neural network** (universal approximation theorem)

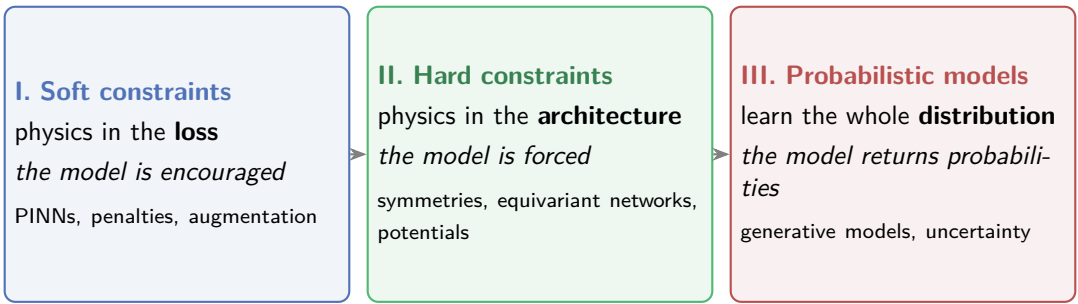


# Generalisation and inductive bias

- Goal: perform well on **new** data, not on the training set
- Split data: **train** / **validation** / **test**
- Overfitting: the model memorises noise
- **Inductive bias**: the assumptions built into the model decide how it generalises, especially **outside the data**
- Physical knowledge is a strong inductive bias



# Three ways to put physics into a model




Prior knowledge reduces the amount of data needed.

# I. Soft constraints: physics in the loss

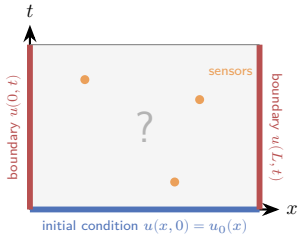
# The idea: add physics to the loss

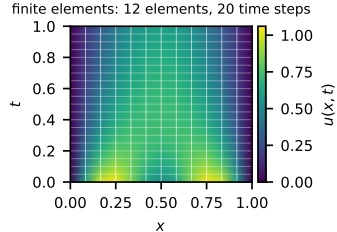
$$\mathcal{L}(\theta) = \underbrace{\frac{1}{N}\sum_i \left(f_\theta(x_i) - y_i\right)^2}_{\text{fit the data}} + \sum_k \lambda_k \underbrace{\mathcal{R}_k[f_\theta]}_{\text{violate the physics}}$$

- $\mathcal{R}_k \geq 0$, zero when the physics is satisfied
- Can be evaluated **anywhere**, not only where we have data
- The network architecture does not change: any model, any optimiser
- Physics acts as a **regulariser**: it pulls the model towards physical solutions
- But the constraint is only satisfied **approximately**, as a trade-off set by $\lambda_k$

# The problem: solving a PDE with a neural network

$$\partial_t u = D\,\partial_x^2 u \quad x \in [0, L],\ t \in [0, T] \qquad u(x, 0) = u_0(x) \qquad u(0, t) = u(L, t) = 0$$

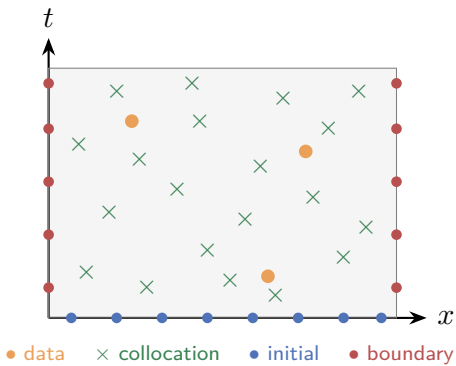



- **Known**: the equation, initial and boundary conditions, a few sensors
- **Unknown**: $u(x, t)$ everywhere inside

- Classical route (right): mesh + finite elements, solve on the grid
- Neural-network route: $u_\theta(x, t)$, mesh-free and differentiable
- But: no labels inside the domain. How do we train it?

# The PINN solution: the equation becomes the training signal

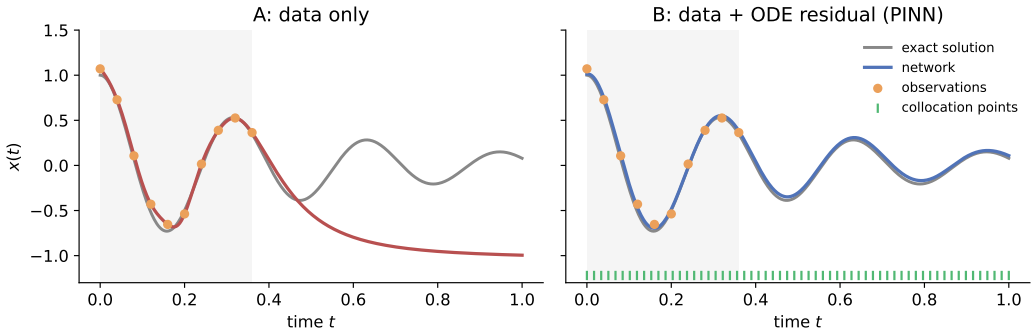



$$\mathcal{L}(\theta) = \sum_i \left|u_\theta(x_i, t_i) - u_i\right|^2 + \frac{1}{N_r}\sum_r \left|\partial_t u_\theta - D\,\partial_x^2 u_\theta\right|^2_{(x_r,t_r)} + \sum_j \left|u_\theta(x_j, 0) - u_0(x_j)\right|^2 + \sum_k \left|u_\theta(x_k^\partial, t_k) - g(t_k)\right|^2$$

- Derivatives of $u_\theta$ by **automatic differentiation**: exact, no mesh
- Collocation points are **free**: no labels needed, put them anywhere
- No data $\Rightarrow$ forward solver; with data $\Rightarrow$ **inverse problem** (e.g. learn $D$ too)

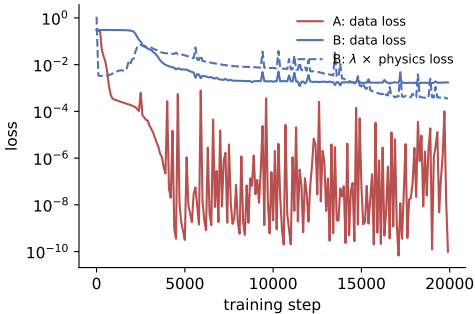
Lagaris, Likas & Fotiadis, IEEE TNN 9, 987 (1998) · Raissi, Perdikaris & Karniadakis, J. Comput. Phys. 378, 686 (2019)

# Demo I: a damped oscillator, observed only at the beginning

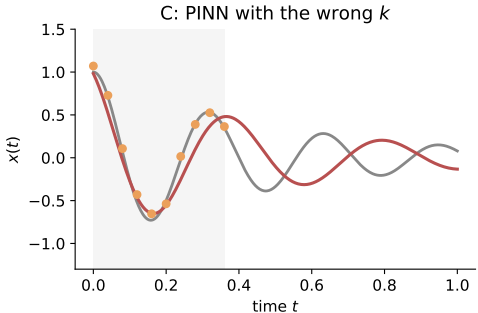



$\ddot{x} + \mu\dot{x} + kx = 0$, 10 noisy points in the grey region. **Same network**, only the loss differs: A: data only $\Rightarrow$ fits the points, wrong afterwards. B: + ODE residual on 60 collocation points $\Rightarrow$ follows the physics everywhere.

# Demo I: two lessons




The data-only model has the **lowest training loss** (it memorises the noise) and the **worst** prediction.




A PINN with the **wrong** stiffness $k$ fits the data and gives a smooth, plausible curve that is wrong afterwards. The result relies on the physics being correct.

# Soft constraints beyond PDEs

### Conservation and sum rules

- total charge $\sum_i q_i = Q$ in ML charge models
- $\int \rho(\mathbf{r})\, d\mathbf{r} = N_{\text{electrons}}$, $f$-sum rule
- probabilities sum to one

### Energy–force consistency

- separate heads for $E$ and $\mathbf{F}$, penalty $\|\mathbf{F} + \nabla E\|^2$

### Known limits

- ideal-gas limit, short-range repulsion, dissociation limit

### Kinetics

- detailed balance $\pi_i k_{ij} = \pi_j k_{ji}$ in learned rate matrices; non-negative rates

### Simulation-based constraints

- the model output goes through a simulation before the loss: $\mathcal{L} = \|\mathrm{Sim}(f_\theta) - y_{\text{meas}}\|^2$
- gradients go through the simulator: **differentiable simulation**
- e.g. adjust a force model until simulated and observed trajectories match

### Shape constraints

- monotonicity, convexity, bounds

Any equation, inequality or limit you trust can become a loss term.

# Two more soft tools

**Data augmentation**

- Known symmetry? Show the model transformed copies of the data (rotated, permuted, translated)
- The model **learns** the symmetry, approximately and only near the data
- The soft version of what Part II does exactly

**Neural operators**

- Learn a map between **functions**: initial condition, parameters $\mapsto$ solution field
- Fourier neural operators, DeepONet (Li et al. 2021; Lu et al. 2021)
- Physics-informed versions add the PDE residual to the loss: one model for a whole family of problems

Li et al., ICLR (2021) · Lu et al., Nat. Mach. Intell. 3, 218 (2021)

# Soft constraints: limitations, and when to use them

**Limitations**

- Choosing the weights $\lambda_k$: competing terms
- Stiff, ill-conditioned loss landscapes; PINNs can fail to train (Krishnapriyan et al. 2021; Wang et al. 2021)
- Spectral bias: slow to learn sharp or high-frequency features
- Constraint satisfied only approximately, only where evaluated
- Wrong physics $\Rightarrow$ wrong model, without warning

**When to use them**

- Known equations, scarce or noisy data
- Inverse problems: infer parameters or a missing term
- Approximate physics you don't want to impose exactly
- Quick to try: only the loss changes

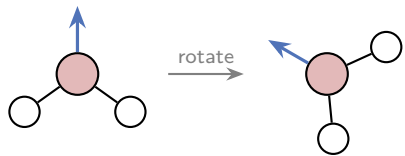
This week: *"What accuracy is lost by removing the physics?"* is one of the questions of the surrogate challenge.

Krishnapriyan et al., NeurIPS (2021) · Wang, Teng & Perdikaris, SIAM J. Sci. Comput. 43, A3055 (2021)

# II. Hard constraints: physics in the architecture

# Symmetries: invariance and equivariance

- Physical laws do not depend on how you label or orient things
  - translations, rotations, permutations of identical particles, time reversal, gauge
- **Invariance**: $f(g \cdot x) = f(x)$
  - energy $E(R\mathbf{r}) = E(\mathbf{r})$
- **Equivariance**: $f(g \cdot x) = g \cdot f(x)$
  - forces $\mathbf{F}(R\mathbf{r}) = R\,\mathbf{F}(\mathbf{r})$
- How do we get a model that satisfies this **exactly**?




$E$ unchanged (invariant)
force rotates (equivariant)

# Four routes to an exact symmetry

**1. Invariant inputs**

$x \;\rightarrow$ distances, angles $\rightarrow$ NN

describe the system only by invariants (symmetry functions, SOAP, ACE)

**2. Canonicalise**

$x \rightarrow$ standard pose $\rightarrow$ NN

align to inertia frame, sort atoms; or learn the frame

**3. Average**

$\bar{f}(x) = \frac{1}{|G|}\sum_g g^{-1} f(gx)$

symmetrise any model over the group or a set of frames

**4. Equivariant layers**

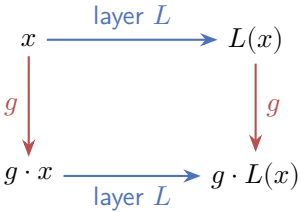
$x \rightarrow$ layer $\rightarrow \cdots \rightarrow$ layer

every layer commutes with the group (CNN, GNN, e3nn)

| | | | |
|---|---|---|---|
| + simple, cheap | any network, exact | any network, exact | most expressive, data-efficient |
| – may lose information (chirality), scalar outputs | discontinuous at degeneracies | cost $\propto$ number of group elements | complex, costly |

Behler & Parrinello, PRL 98, 146401 (2007) · Kaba et al., ICML (2023) · Puny et al., ICLR (2022) · Cohen & Welling, ICML (2016) · Zaheer et al., NeurIPS (2017) · Satorras et al., ICML (2021)

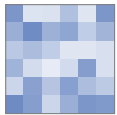
# Equivariance is a constraint on the architecture



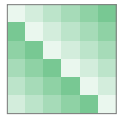

Transform then apply = apply then transform.

- Impose it **layer by layer**: equivariant layers compose into an equivariant network
- Finish with a sum or mean $\Rightarrow$ invariant output
- The constraint acts on the **weights**: only a subspace of weight matrices is allowed

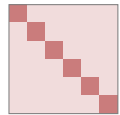
linear layer
$L(x) = W \cdot x$
($n$ inputs)



no symmetry
$n^2$ free weights



translations
$n$ weights: a **convolution**



permutations
2 weights: $W = a\, I + b\, \mathbf{1}\mathbf{1}^\top$

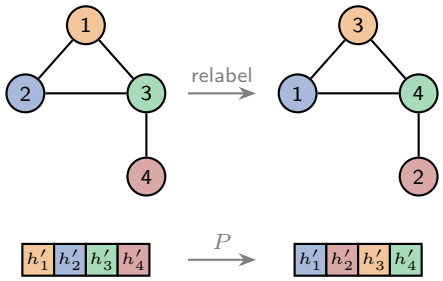
Cohen & Welling, ICML (2016) · Zaheer et al., NeurIPS (2017) · Kondor & Trivedi, ICML (2018)

# Which symmetry, which architecture?

| Symmetry | Typical data | Architecture | What the constraint does |
|---|---|---|---|
| translations | images, grids, time series | **CNN** | same filter everywhere |
| permutations (sets) | particles, point clouds | **Deep Sets, transformer** | same function per element, order-blind pooling |
| permutations (graphs) | molecules, meshes | **GNN** | same message on every edge |
| E(3): rotations + translations | atoms, 3D point clouds | **EGNN, NequIP, MACE** | vector/tensor features, tensor products |
| rotations on the sphere | climate, cosmology maps | **spherical CNN** | convolution on the sphere |
| Lorentz group | particle jets | **Lorentz-equivariant** | four-vectors, Minkowski products |
| gauge symmetry | lattice field theory | **gauge-equivariant** | built from gauge-invariant loops |

Cohen et al., ICLR (2018) · Satorras et al., ICML (2021) · Bogatskiy et al., ICML (2020) · Kanwar et al., PRL 125, 121601 (2020)

# Example: a graph neural network is permutation-equivariant




same atoms, new labels:
the outputs are reordered the same way

$$h'_i = \phi\Big(h_i,\ \textstyle\sum_{j\in\mathcal{N}(i)} \psi(h_i, h_j)\Big)$$

- Same $\phi$, $\psi$ on every node and edge; a **sum** does not depend on the order of the neighbours
- Relabel the nodes: $H \to PH$, adjacency $A \to PAP^\top$

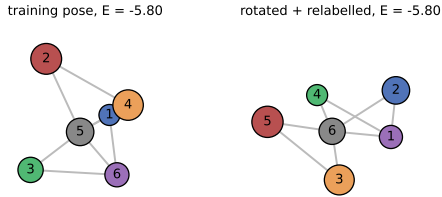
$$F(PH,\ PAP^\top) = P\,F(H, A)$$

- The layer is **equivariant**: each atom keeps its own output (charge, force)
- **Invariance** comes only at the end: $E = \sum_i \rho(h_i)$

Gilmer et al., ICML (2017) · Zaheer et al., NeurIPS (2017) · Bronstein et al., arXiv:2104.13478

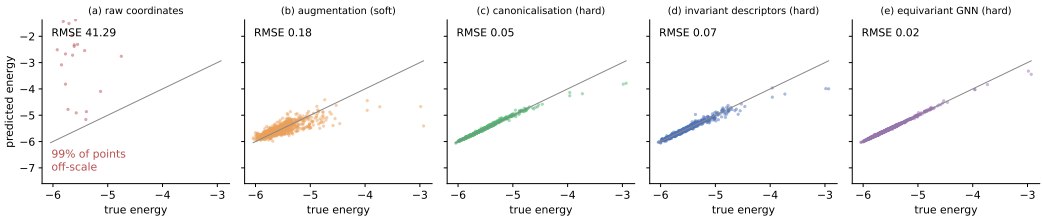
# Demo II: one problem, five models

- Energy of a 6-atom cluster (Lennard-Jones + three-body term)
- Trained in **one orientation**, tested on **rotated**, then relabelled, copies
- Same cluster, same energy, 18 different input numbers



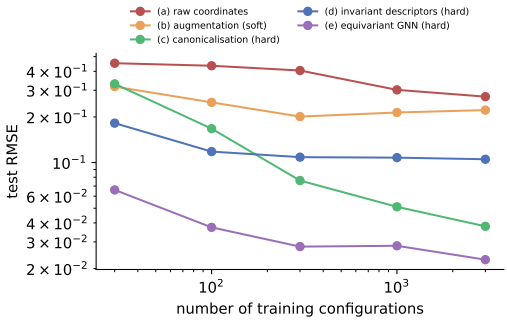

| | Input | Network |
|---|---|---|
| **(a)** raw coordinates | 18 coordinates | MLP, $2 \times 64$ hidden units |
| **(b)** augmentation (soft) | 18 coordinates, randomly rotated at each step | same MLP, $10\times$ more training steps |
| **(c)** canonicalisation | 18 coordinates in a standard pose (inertia axes, sorted atoms) | same MLP |
| **(d)** invariant descriptors | 28 distance and angle functions per atom | same MLP per atom, summed |
| **(e)** equivariant GNN | positions; messages along interatomic vectors | one message-passing layer, summed over atoms |

# Demo II: five ways to handle symmetry

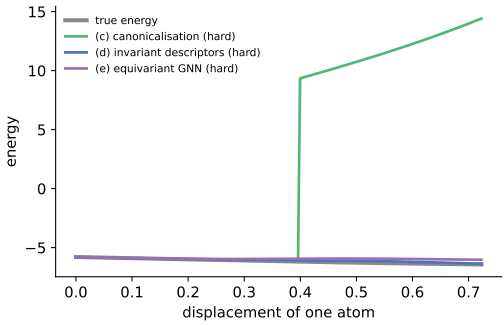



- (a) raw coordinates: off-scale on rotated data
- (b) augmentation: learns the rotations, but with $10\times$ more training; relabel the atoms and it fails
- (c), (d), (e): $E(x) = E(gx)$ to $10^{-14}$, by construction; the equivariant network is the most accurate

# Demo II: data efficiency and the price of each route




Training data now in random orientations.




Move one atom smoothly: two atoms swap places in the sorting order and the canonicalised model **jumps**.

# Hard constraints beyond symmetry

| Route | Example |
|---|---|
| derive from a potential | $\mathbf{F} = -\nabla E_\theta$: conservative forces; $\mathbf{B} = \nabla \times \mathbf{A}_\theta$: divergence-free; Hamiltonian / Lagrangian NNs conserve energy |
| output parametrisation | softplus $\Rightarrow$ positive; softmax $\Rightarrow$ normalised; $LL^\top \Rightarrow$ positive-definite matrix; input-convex NNs $\Rightarrow$ convex |
| projection layer | $q_i \leftarrow q_i - (\sum_j q_j - Q)/N$: exact total charge; optimisation layers (OptNet) |
| trial-function ansatz | $u = g(x) + d(x)\,\mathrm{NN}(x)$: boundary conditions exact (Lagaris et al. 1998) |
| locality, extensivity | $E = \sum_i \varepsilon(\text{environment}_i)$: size-extensive, linear scaling |
| hybrid model | keep the known solver, learn only the missing term (universal differential equations) |
| normalised by construction | normalizing flows: exact probability densities ($\rightarrow$ Part III) |

Greydanus et al., NeurIPS (2019) · Cranmer et al., arXiv:2003.04630 · Amos et al., ICML (2017) · Lagaris et al., IEEE TNN 9, 987 (1998) · Rackauckas et al., arXiv:2001.04385

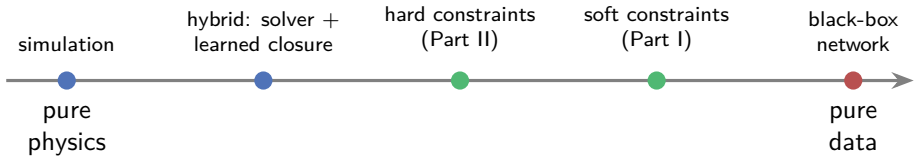
# Hard constraints: limitations

- You must know the constraint **exactly**
- **Broken** symmetries: external fields, boundaries, finite-size effects
- Canonicalisation: discontinuities at degenerate configurations
- Invariant descriptors: information loss
- Expressivity and computational cost of equivariant layers
- **Over-constraining**: no room left to learn what the physics misses

# Soft vs hard

| | **Soft** (Part I) | **Hard** (Part II) |
|---|---|---|
| how | penalty in the loss, augmentation | built into the model |
| guarantee | approximate, near the data | exact, **everywhere** |
| data needed | more | less |
| flexibility | any architecture | needs a suitable design |
| if the physics is wrong | data can partly correct it | the error cannot be corrected |

# Where we stand: a spectrum




- The further to the right, the more data you need and the less you can extrapolate
- Most successful AI-for-science models sit in the **middle**
- So far, every model returns **one answer**. What are the other plausible ones, and how much should we trust it? → Part III

# III. Generative and probabilistic models

# Why generative models in science?

- Parts I and II: learn a **function**, one input $\rightarrow$ one answer
- Now: learn a **distribution** $p(x)$ from examples, the set of *plausible* objects
- Scientific spaces are huge and sparse
  - $\sim 10^{60}$ drug-like molecules; crystals; protein sequences; possible signals
- A generative model learns where the interesting part is, and lets us **explore** it

**Generate**: propose new candidates
materials, proteins, simulated events

**Condition**: all answers compatible with what is known
a target property, a measurement $\Rightarrow$ **inverse problems**, **uncertainty**

**Score**: how plausible is this one?
low probability $\Rightarrow$ **anomaly**, out of distribution

MatterGen: Nature 639, 624 (2025) · RFdiffusion: Nature 620, 1089 (2023) · Paganini et al., PRL 120, 042003 (2018)

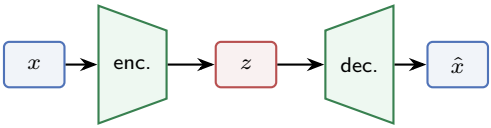
# A family of generative models

Common idea: draw from a **known distribution**, learn a **map** to the data distribution.

$$z \sim \mathcal{N}(0,1) \xrightarrow{\text{learned map } G_\theta} x = G_\theta(z), \qquad x \sim p_\theta(x) \approx p_{\text{data}}(x)$$

| Model | Idea | $p(x)$? |
|---|---|---|
| **VAE** | map from a few Gaussian latent variables | lower bound |

**Autoencoder (VAE)**




Typical use: compact maps, anomaly detection

- All can be **conditioned**: $p(x \mid \text{what we know})$. The demo uses the autoencoder

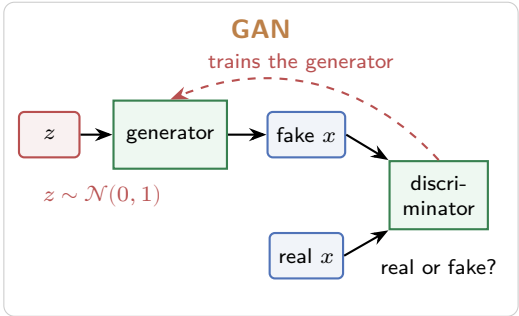
Kingma & Welling, ICLR (2014) · Goodfellow et al., NeurIPS (2014) · Rezende & Mohamed, ICML (2015) · Ho et al., NeurIPS (2020)

# A family of generative models

Common idea: draw from a **known distribution**, learn a **map** to the data distribution.

$$z \sim \mathcal{N}(0,1) \xrightarrow{\text{learned map } G_\theta} x = G_\theta(z), \qquad x \sim p_\theta(x) \approx p_{\text{data}}(x)$$

| Model | Idea | $p(x)$? |
|---|---|---|
| **VAE** | map from a few Gaussian latent variables | lower bound |
| **GAN** | map trained to fool a discriminator | no |




Typical use: fast simulation (calorimeter showers)

- All can be **conditioned**: $p(x \mid \text{what we know})$. The demo uses the autoencoder

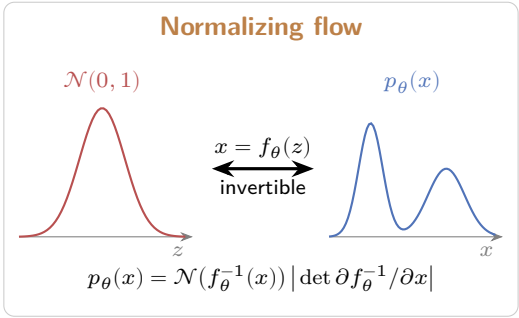
Kingma & Welling, ICLR (2014) · Goodfellow et al., NeurIPS (2014) · Rezende & Mohamed, ICML (2015) · Ho et al., NeurIPS (2020)

# A family of generative models

Common idea: draw from a **known distribution**, learn a **map** to the data distribution.

$$z \sim \mathcal{N}(0,1) \xrightarrow{\text{learned map } G_\theta} x = G_\theta(z), \qquad x \sim p_\theta(x) \approx p_{\text{data}}(x)$$

| Model | Idea | $p(x)$? |
|---|---|---|
| **VAE** | map from a few Gaussian latent variables | lower bound |
| **GAN** | map trained to fool a discriminator | no |
| **Flow** | the map is invertible | exact |




Typical use: exact densities, sampling, inference

- All can be **conditioned**: $p(x \mid \text{what we know})$. The demo uses the autoencoder

Kingma & Welling, ICLR (2014) · Goodfellow et al., NeurIPS (2014) · Rezende & Mohamed, ICML (2015) · Ho et al., NeurIPS (2020)

# A family of generative models

Common idea: draw from a **known distribution**, learn a **map** to the data distribution.

$$z \sim \mathcal{N}(0,1) \xrightarrow{\text{learned map } G_\theta} x = G_\theta(z), \qquad x \sim p_\theta(x) \approx p_{\text{data}}(x)$$

| Model | Idea | $p(x)$? |
|---|---|---|
| **VAE** | map from a few Gaussian latent variables | lower bound |
| **GAN** | map trained to fool a discriminator | no |
| **Flow** | the map is invertible | exact |
| **Diffusion** | the map is many small denoising steps | lower bound |

**Diffusion model**

add a little noise at each step (fixed)




remove it, step by step (**learned**)

generate: start from noise, denoise $T$ times

Typical use: images, molecules, weather ensembles

- All can be **conditioned**: $p(x \mid \text{what we know})$. The demo uses the autoencoder

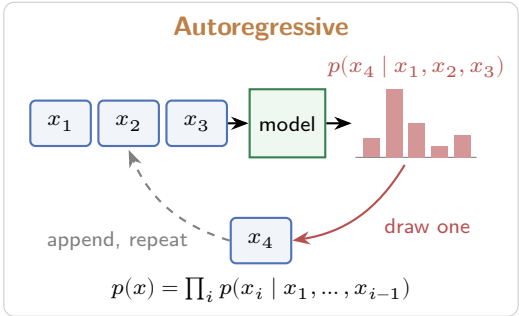
Kingma & Welling, ICLR (2014) · Goodfellow et al., NeurIPS (2014) · Rezende & Mohamed, ICML (2015) · Ho et al., NeurIPS (2020)

# A family of generative models

Common idea: draw from a **known distribution**, learn a **map** to the data distribution.

$$z \sim \mathcal{N}(0,1) \xrightarrow{\text{learned map } G_\theta} x = G_\theta(z), \qquad x \sim p_\theta(x) \approx p_{\text{data}}(x)$$

| Model | Idea | $p(x)$? |
|---|---|---|
| **VAE** | map from a few Gaussian latent variables | lower bound |
| **GAN** | map trained to fool a discriminator | no |
| **Flow** | the map is invertible | exact |
| **Diffusion** | the map is many small denoising steps | lower bound |
| **Autoregressive** | one element at a time, drawn from a learned discrete distribution | exact |




Typical use: text, sequences, LLMs

- All can be **conditioned**: $p(x \mid \text{what we know})$. The demo uses the autoencoder

Kingma & Welling, ICLR (2014) · Goodfellow et al., NeurIPS (2014) · Rezende & Mohamed, ICML (2015) · Ho et al., NeurIPS (2020)

# Putting what we know into a generative model

The methods of Parts I and II also apply to generative models.

**Soft** (Part I)

- Penalise generated samples that violate an equation or a conservation law
- Use a known energy or a simulator as a training signal
- **Guidance**: steer the sampling towards a constraint or a target property

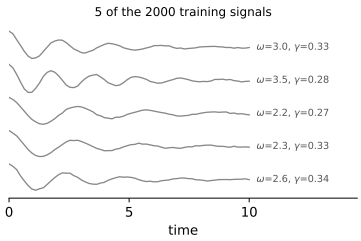
**Hard** (Part II)

- Equivariant generator $\Rightarrow p(x)$ has the symmetry: a rotated molecule is equally likely
- Valid by construction: positivity, periodicity, fixed composition
- Generate in a physical parametrisation, then map to the data

Examples: equivariant diffusion for molecules, energy-trained flows, physics-guided weather ensembles (GenCast).

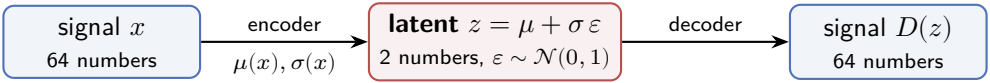
Hoogeboom et al., ICML (2022) · Noé et al., Science 365, eaaw1147 (2019) · Price et al., Nature 637, 84 (2025)

# Demo III: a variational autoencoder on signals




2000 damped oscillations $x(t) = e^{-\gamma t}\cos(\omega t)$, 64 samples each, plus noise (0.02). $\omega$ and $\gamma$ are drawn **uniformly**: $\omega \in [2, 4]$, $\gamma \in [0.1, 0.4]$.

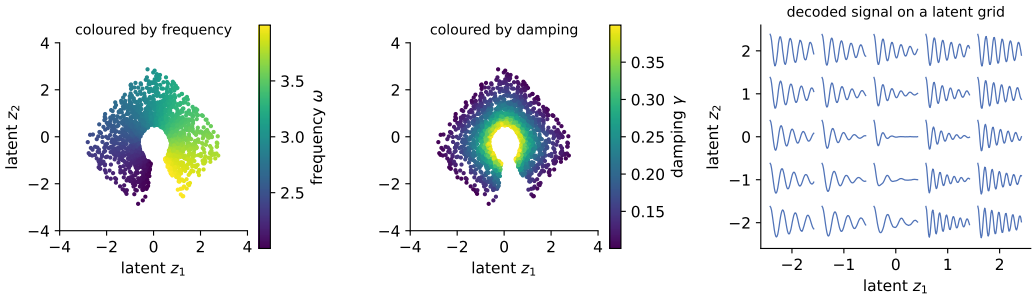
- **Encoder**: MLP $64 \to 256 \to 256 \to 4$. It returns a mean $\mu(x)$ and a width $\sigma(x)$ for each of the 2 latent variables
- **Decoder** $D$: MLP $2 \to 256 \to 256 \to 64$
- A **standard VAE**, trained on the signals alone: 12000 Adam steps, 20 s on a laptop




$$\mathcal{L} = \underbrace{\frac{\|x - D(z)\|^2}{2\sigma_d^2}}_{\text{reconstruct the data } (\sigma_d = 0.1)} + \underbrace{\mathrm{KL}(\mathcal{N}(\mu, \sigma^2) \,\|\, \mathcal{N}(0,1))}_{\text{latent points of the data} \sim \mathcal{N}(0,1)}$$

To generate: draw $z \sim \mathcal{N}(0,1)$, decode. The decoder is the map from $\mathcal{N}(0,1)$ to the data.

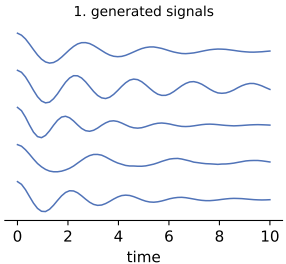
# Demo III: the latent space is a map of the signals




The two latent numbers form a (distorted) **map of frequency and damping**, although the model was never given these parameters. Moving in the latent space changes the signal smoothly: this is the space we explore.

# Demo III: generate, then check the physics

The model was given no physics. So we test what it generates against what we know.



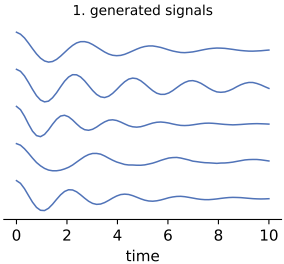

**Generate**

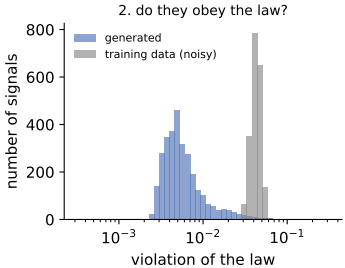
Draw $z \sim \mathcal{N}(0, 1)$, decode.

The signals look right. Are they?

# Demo III: generate, then check the physics

The model was given no physics. So we test what it generates against what we know.




## Generate

Draw $z \sim \mathcal{N}(0, 1)$, decode.

The signals look right. Are they?

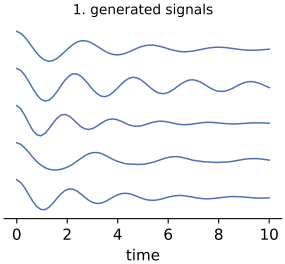
## Check 1: the law

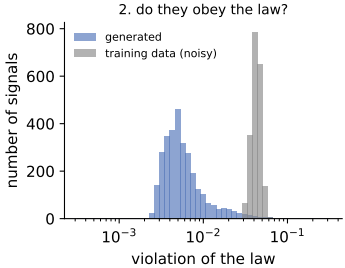
How far is each signal from a solution of $\ddot{x} + 2\gamma\,\dot{x} + \omega_0^2\,x = 0$, whatever $\omega$ and $\gamma$?

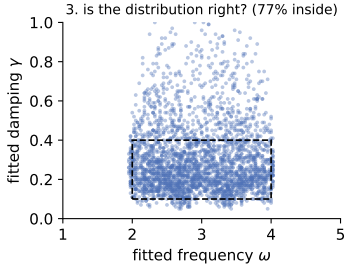
**Yes**: 8 times closer than the noisy data (0.005 against 0.04).

# Demo III: generate, then check the physics

The model was given no physics. So we test what it generates against what we know.




### Generate

Draw $z \sim \mathcal{N}(0, 1)$, decode.

The signals look right. Are they?

### Check 1: the law

How far is each signal from a solution of $\ddot{x} + 2\gamma\,\dot{x} + \omega_0^2\,x = 0$, whatever $\omega$ and $\gamma$?

**Yes**: 8 times closer than the noisy data (0.005 against 0.04).

### Check 2: the distribution

Fit $\omega, \gamma$ to each signal. Are they in the training ranges (dashed)?

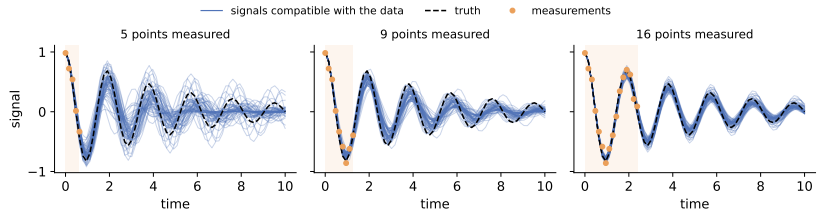
**Partly**: 77% are. The others come from the hole in the latent map.

The **form** of the signals is learned well, their **distribution** only approximately.

# Uncertainty: the generative model as a prior

A few noisy measurements $y$ of a signal. What is the rest? Every signal that is

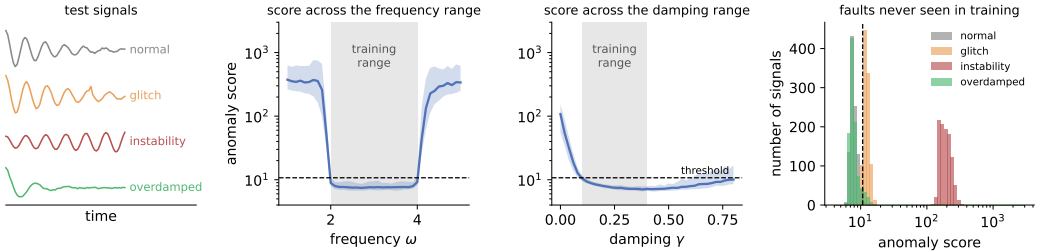
$$p(x \mid y) \propto \underbrace{p(y \mid x)}_{\text{compatible with the data}} \times \underbrace{p(x)}_{\text{physical: the generative model}}$$




- The **prior** $p(x)$ can be **pretrained** once (simulations, archives) and reused
- Spread of the samples = **uncertainty**
- **Limits**: depends on the prior; unreliable for cases unlike the training data

# Anomaly detection: low-probability events

Train on **normal** data only; score a new signal by $-\ln p(x)$. No example of a fault is needed.




- The score finds the **edge of the training data** in frequency; 93–100% of the faults are flagged
- Out of distribution does not mean faulty: $\omega = 4.5$ is a valid oscillation
- Pitfall: overdamped signals, outside the training range, pass as normal (7% flagged)

Nalisnick et al., ICLR (2019) · Chandola, Banerjee & Kumar, ACM Comput. Surv. 41, 15 (2009)

# Probabilistic models: links and pitfalls

## Links to this week

- **Anomaly detection**: low likelihood under a model of "normal" $\Rightarrow$ suspicious
- **Optimisation**: uncertainty decides where to evaluate next (Bayesian optimisation)
- **Surrogates**: error bars tell you when to stop trusting the model (out of distribution)

## Limitations

- Generative models are not easy to evaluate: quality of samples is not enough
- Mode collapse; rare events under-represented
- Overconfidence out of distribution
- Uncertainty that is itself not calibrated

## Other ways to get an uncertainty

- **Ensembles**: train several models, use their spread; **Gaussian processes**; Bayesian neural networks, MC dropout
- Predict a width $\sigma_\theta(x)$ together with the value
- **Conformal prediction**: intervals calibrated on held-out data, for any model

# Conclusion

# Three ways to put physics into a model

| | **encodes** | **guarantee** | **tools** | **use when** |
|---|---|---|---|---|
| **I. Soft** | equations, limits, sum rules in the loss | approximate, near the evaluated points | PINNs, penalties, augmentation | physics known but approximate; few data |
| **II. Hard** | symmetries and structure in the model | exact, everywhere | invariant features, canonicalisation, equivariant nets, potentials | constraint known exactly |
| **III. Probabilistic** | the full distribution | error bars (if calibrated) | GPs, ensembles, flows, diffusion | you need uncertainty or samples |

# Resources

**Software**

- General: PyTorch, JAX, scikit-learn
- Physics-informed: DeepXDE, neuraloperator
- Symmetry: PyTorch Geometric, e3nn, MACE
- Uncertainty: GPyTorch, BoTorch
- Generative: sbi (simulation-based inference)

**Demos of this lecture**: `demos/` (three notebooks, CPU, 25 s to 3 min each)

**To go further**

- *Physics-based Deep Learning* (Thuerey et al., online book)
- *Geometric Deep Learning* (Bronstein et al., 2021)
- *Deep Learning* (Goodfellow, Bengio, Courville)

**Next: Accelerators and AI**

# Questions?

Have a great hackathon!